

LegalEase AI

AI-Powered Legal Document Drafting Assistant

LegalEase AI

AI-powered legal document generation

About

- Employment Agreement
- NDA
- Lease Agreement
- Service Agreement
- Partnership Agreement

Important

Drafting assistance only.

Create Your Legal Document

Document Type

Employment Agreement

Parties

Alice Smith (Employee) / ABC Technologies (Employer)

Effective Date

September 29, 2026

Terms and Conditions

Confidential information must be protected; 30-day notice; monthly payment

Generate Legal Document

Generated draft → Edit → Preview → Download TXT / DOCX / PDF

Based on the supplied LegalEaseAI 2 source archive and LegalEase sample document

1. Executive Summary

LegalEase AI is a web-based application designed to turn user-supplied legal facts and terms into editable legal-document drafts. The supplied implementation combines a Streamlit frontend, a FastAPI backend, a Gemini-based generation layer, sanitization utilities, and export services for TXT, DOCX, and PDF.

Area	Implementation observed
Frontend	Streamlit web interface
Backend	FastAPI REST API
AI generation	Google Gemini via google-genai
Validation	Pydantic request/response schemas
Exports	TXT, DOCX, PDF
Deployment	Dockerfile, docker-compose, Procfile
Testing	Pytest + FastAPI TestClient

Project objective

Reduce the effort required to prepare a first legal-document draft while keeping user-provided facts explicit, missing information visible as [TO BE COMPLETED], and the result editable before export.

2. Product Highlights

- Guided legal-document input with predefined document types and a custom “Other” option.
- AI-assisted drafting with a dedicated system instruction that emphasizes fact preservation and non-fabrication.
- Input validation and generated-output size protection.
- Fallback Gemini model list plus retry/backoff handling for temporary availability errors.
- In-app editing and preview before the user exports the final draft.
- Multi-format export: TXT, DOCX and PDF.
- Deployment-ready files for Docker and Procfile-based hosting.
- Automated API tests for root, health and demo generation paths.

 **LegalEase AI**

AI-powered legal document generation

About

- Employment Agreement
- NDA
- Lease Agreement
- Service Agreement
- Partnership Agreement

Important

Drafting assistance only.

Create Your Legal Document

Document Type

Employment Agreement

Parties

Alice Smith (Employee) / ABC Technologies (Employer)

Effective Date

September 29, 2026

Terms and Conditions

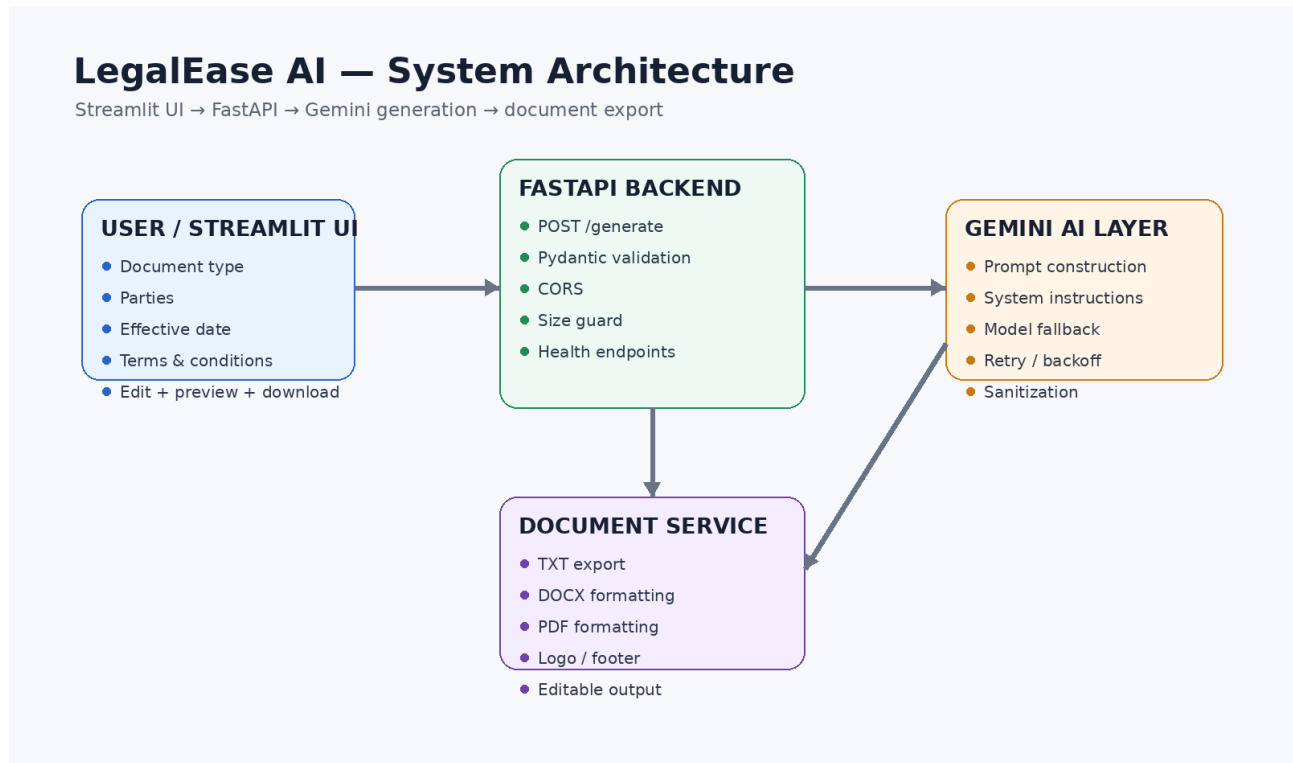
Confidential information must be protected; 30-day notice; monthly payment

Generate Legal Document

Generated draft → Edit → Preview → Download TXT / DOCX / PDF

3. System Architecture

The codebase follows a clear separation between presentation, API orchestration, AI generation, document formatting, and utilities.



Architecture note: the supplied project is primarily a single-repository application. The Streamlit frontend calls the FastAPI backend at <http://127.0.0.1:8000>, while document formatting functions are imported directly by the frontend for downloads.

4. End-to-End Workflow

LegalEase AI — Document Lifecycle



Generation modes

Demo mode: when DEMO_MODE is enabled, the generator returns deterministic template-style output without calling Gemini. This is useful for demonstrations and deterministic automated tests.

Production mode: when demo mode is disabled, the backend requires GEMINI_API_KEY, builds a structured prompt, calls the configured Gemini model, sanitizes the returned text, and returns the document through the API.

5. Frontend Documentation

The Streamlit interface is implemented in frontend/app.py. The page is configured with a wide layout and contains a sidebar, input area, generation button, editable result area, preview, download controls, and a legal disclaimer.

- **Document type:** Employment Agreement, NDA, Lease Agreement, Service Agreement, Partnership Agreement, Freelance Agreement, Sales Agreement, Consulting Agreement, or Other.
- **Parties:** free-text participant information.
- **Effective Date:** free-text date field.
- **Terms and Conditions:** user-provided clauses/requirements.
- **Editor:** generated text is placed into an editable text area.
- **Preview:** the current edited document is displayed before download.
- **Download:** TXT, DOCX and PDF buttons are provided.

The screenshot displays the 'LegalEase AI' web application. The header is dark blue with the logo on the left and the text 'AI-powered legal document generation' on the right. The main content area is white and titled 'Create Your Legal Document'. It features a sidebar on the left with an 'About' section listing document types and an 'Important' section with a disclaimer. The main form has five input fields: 'Document Type' (set to 'Employment Agreement'), 'Parties' (set to 'Alice Smith (Employee) / ABC Technologies (Employer)'), 'Effective Date' (set to 'September 29, 2026'), and 'Terms and Conditions' (set to 'Confidential information must be protected; 30-day notice; monthly payment'). A large blue button labeled 'Generate Legal Document' is positioned below the form. At the bottom, a navigation bar shows the workflow: 'Generated draft → Edit → Preview → Download TXT / DOCX / PDF'.

 **LegalEase AI**

AI-powered legal document generation

About

- Employment Agreement
- NDA
- Lease Agreement
- Service Agreement
- Partnership Agreement

Important

Drafting assistance only.

Create Your Legal Document

Document Type

Employment Agreement

Parties

Alice Smith (Employee) / ABC Technologies (Employer)

Effective Date

September 29, 2026

Terms and Conditions

Confidential information must be protected; 30-day notice; monthly payment

Generate Legal Document

Generated draft → Edit → Preview → Download TXT / DOCX / PDF

LegalEase AI • Project Documentation • Page 6

6. Backend & API Documentation

FastAPI is initialized in backend/main.py. CORS is configured from the environment, the routes are included, and two health-style endpoints are exposed.

Endpoints

Method	Endpoint	Purpose
GET	/	Returns status=ok and app name.
GET	/health	Health check with the application name.
POST	/generate	Generates a legal-document draft from validated input.

POST /generate input

document_type: 2–120 characters; parties: 2–4000; terms: 2–12000; effective_date: 2–100. Each value is stripped and rejected if empty.

POST /generate output

Returns document, model and demo_mode. Generated output is rejected with HTTP 413 when it exceeds the configured maximum document size.

7. AI Generation Layer

The GeminiDocumentGenerator in backend/ai_core/gemini_generator.py is responsible for AI-assisted drafting. Its system instruction explicitly tells the model not to invent names, dates, addresses, monetary values, governing laws, legal citations, or obligations. Missing information should be represented with [TO BE COMPLETED].

Prompt construction

The request is transformed into a structured prompt containing the document type, parties, effective date, and semicolon-separated user terms converted into bullet-like lines. The model is asked to use clear headings, numbered clauses, preserve every supplied term, avoid fabricated facts, and include signature blocks.

Resilience

Production mode tries the configured model followed by fallback model names present in the source. Temporary 503/unavailable/high-demand style errors trigger up to three attempts with exponential waits of 2, 4 and 8 seconds before moving on.

Safety-oriented behavior in the supplied code

The generator identifies the application as drafting assistance rather than legal advice. This is consistent with the frontend disclaimer and the sample document workflow, but users still need appropriate professional review before relying on generated text.

8. Document Export & Formatting

backend/services/document_service.py provides three output paths:

- **TXT:** UTF-8 encoded text.
- **DOCX:** Times New Roman styling, centered LegalEase logo/header, heading detection, bullet handling, optional key-terms table, and footer text.
- **PDF:** FPDF-based legal-document layout with LegalEase header, Times font, page numbering, and footer.

The supplied sample document demonstrates the resulting legal-document structure: title, effective date, parties, numbered clauses, governing-law section, entire-agreement clause, and signature blocks.

9. Supplied Sample Document — Visual Evidence

The uploaded PDF is a three-page Employment Agreement generated with LegalEase AI. It contains placeholders for missing facts, including employer/employee addresses, position, working days, payment schedule, governing law and jurisdiction. The sample also shows salary, confidentiality, termination, entire-agreement and signature sections.

LegalEase AI

LegalEase AI Legal Document

EMPLOYMENT AGREEMENT

This Employment Agreement (the "Agreement") is made and entered into as of October 1, 2026 (the "Effective Date"), by and between:

EMPLOYER:

ABC Technologies Pvt Ltd, with its principal place of business located at [TO BE COMPLETED] (hereinafter referred to as the "Employer"),

AND

EMPLOYEE:

John Doe, residing at [TO BE COMPLETED] (hereinafter referred to as the "Employee").

The Employer and Employee may collectively be referred to as the "Parties" or individually as a "Party."

RECITALS

WHEREAS, the Employer desires to employ the Employee, and the Employee desires to be employed by the Employer, pursuant to the terms and conditions set forth herein;

NOW, THEREFORE, in consideration of the mutual covenants and promises contained herein, the Parties agree as follows:

1. POSITION AND DUTIES

1.1 Position: The Employee shall serve in the position of [TO BE COMPLETED].

1.2 Duties: The Employee agrees to perform the duties and responsibilities associated with such position as assigned by the Employer.

2. WORKING HOURS

LegalEase AI

2.1 The standard working hours for the Employee shall be from 9:00 AM to 6:00 PM, [TO BE COMPLETED: specify days of the week, e.g., Monday through Friday].

3. COMPENSATION

3.1 Salary: The Employer shall pay the Employee a monthly salary of INR 50,000 (Indian Rupees Fifty Thousand).

3.2 Payment Terms: Such salary shall be paid in accordance with the Employer's standard payroll practices on [TO BE COMPLETED: specify payment schedule/dates].

4. CONFIDENTIALITY

4.1 The Employee agrees that confidentiality must be maintained regarding all proprietary, non-public, and confidential information of the Employer acquired during the course of employment.

4.2 The Employee shall not disclose, directly or indirectly, any such confidential information to any third party without the prior written consent of the Employer, except as required by law.

5. TERMINATION

5.1 Notice Period: Either Party may terminate this Agreement by providing 30 days written notice to the other Party.

5.2 Effect of Termination: Upon termination of employment, the Employee shall return all property belonging to the Employer and fulfill all outstanding obligations through the effective date of termination.

6. GOVERNING LAW AND JURISDICTION

6.1 This Agreement shall be governed by and construed in accordance with the laws of [TO BE COMPLETED].

6.2 Any disputes arising out of or in connection with this Agreement shall be subject to the jurisdiction of the courts located in [TO BE COMPLETED].

7. ENTIRE AGREEMENT

7.1 This Agreement constitutes the entire understanding between the Parties concerning the subject matter hereof and supersedes all prior agreements, representations, or understandings, whether written or

LegalEase AI

oral.

7.2 Any amendment or modification to this Agreement must be in writing and signed by both Parties.

IN WITNESS WHEREOF, the Parties hereto have executed this Employment Agreement as of the Effective Date written above.

EMPLOYER:

ABC Technologies Pvt Ltd

By: _____

Name: [TO BE COMPLETED]

Title: [TO BE COMPLETED]

Date: _____

EMPLOYEE:

John Doe

Signature: _____

Name: John Doe

Date: _____

10. Project Structure

```
LegalEaseAI 2/  
├── backend/  
│   ├── ai_core/gemini_generator.py  
│   ├── services/document_service.py  
│   ├── utils/sanitize.py  
│   ├── config.py  
│   ├── main.py  
│   ├── routes.py  
│   └── schemas.py  
├── frontend/app.py  
├── tests/  
│   ├── conftest.py  
│   └── test_api.py  
├── assets/logo.png  
├── requirements.txt  
├── Dockerfile  
├── docker-compose.yml  
├── Procfile  
├── .env.example  
└── .gitignore
```

Responsibilities

backend/config.py	Environment-backed application settings.
backend/main.py	FastAPI app, CORS and health endpoints.
backend/routes.py	Document-generation API route.
backend/schemas.py	Validated request/response models.
backend/ai_core/gemini_generator.py	Demo/production generation, prompting, retries and sanitization.
backend/services/document_service.py	TEXT/DOCX/PDF formatting.
backend/utils/sanitize.py	Unicode/control-character normalization and term splitting.
frontend/app.py	User interface, API call, editor, preview and downloads.
tests/test_api.py	API smoke/integration-style tests.

11. Configuration & Deployment

The supplied `.env.example` documents the main runtime variables: `GEMINI_API_KEY`, `GEMINI_MODEL`, `DEMO_MODE`, `API_HOST`, `API_PORT`, `LEGALEASE_API_URL`, `CORS_ORIGINS` and `MAX_DOCUMENT_CHARS`.

Local run

1. Create and activate a Python virtual environment.
2. Install dependencies from `requirements.txt`.
3. Copy `.env.example` to `.env` and set the required values.
4. Start FastAPI with uvicorn backend.`main:app --host 0.0.0.0 --port 8000`.
5. Start Streamlit with `streamlit run frontend/app.py`.

Container deployment

The Dockerfile uses `python:3.12-slim`, installs requirements, copies the project, exposes port 8000 and starts Uvicorn. `docker-compose.yml` supplies the `.env` file and maps port 8000. The Procfile provides a web process command suitable for Procfile-based hosting.

Configuration caution

12. Testing & Verification

The source includes tests for the root endpoint, health endpoint and demo-mode generation. The intended assertions verify HTTP 200 responses, application status, and preservation of supplied NDA terms.

Observed verification result

When the supplied archive was executed in the current analysis environment, pytest stopped during test collection because the installed Python environment could not import google.genai. The project requirements.txt does declare google-genai>=1.0,<2.0. Therefore, this is recorded as an environment/dependency-installation issue rather than a claim that the test logic itself is incorrect.

Recommended verification after dependency installation

Run `pytest -q`, then start the backend and exercise GET /, GET /health and POST /generate. Finally, open the Streamlit UI and verify generation, editing, preview and all three download formats.

13. Strengths Visible in the Supplied Implementation

- ✓ Clear separation between UI, API, AI core, document services and utilities.
- ✓ Strong input constraints using Pydantic.
- ✓ Explicit anti-fabrication instructions in the AI system prompt.
- ✓ Demo mode supports deterministic demonstrations without an API call.
- ✓ Temporary AI availability failures receive retries with backoff.
- ✓ Generated text is sanitized before being returned.
- ✓ Users can edit the generated draft before export.
- ✓ Multiple export formats make the result practical for document workflows.
- ✓ Docker and Procfile deployment artifacts are included.

14. Current Limitations / Risks to Address

- The frontend hard-codes `API_URL` to `http://127.0.0.1:8000` rather than reading `LEGALEASE_API_URL` from the environment shown in `.env.example`.
- The sample document and demo generator are intentionally generic; production legal validity still depends on jurisdiction, facts and professional review.
- No authentication or user-account layer is present in the supplied source.
- No persistence/database layer is present; generated documents live in Streamlit session state until the session changes.
- The frontend preview inserts generated text into HTML using `st.markdown`; if future inputs contain HTML-like content, output handling should be reviewed carefully.
- The repository includes a bundled `.venv` and cache/compiled files, which makes the archive larger than necessary for source distribution.

15. Recommended Future Enhancements

Environment-driven API URL: Use LEGALEASE_API_URL consistently so local, Docker and hosted deployments can point to different backend locations.

Authentication & access control: Add user accounts, role-based access and protected document history for real deployments.

Document history: Persist drafts and revisions in a database with timestamps and versioning.

Template library: Add reusable jurisdiction-aware templates with configurable clauses.

Structured review: Add a clause checklist that flags missing placeholders and asks the user to confirm critical facts.

Observability: Add structured logs, request IDs, latency metrics and AI failure monitoring.

Security hardening: Review HTML rendering, secret handling, rate limiting, file handling and CORS policy for production.

Evaluation suite: Add regression tests for prompt adherence, missing-field behavior, output size, export integrity and AI fallback behavior.

16. Final Project Summary

LegalEase AI is structured as a practical AI-document drafting prototype with a clean separation of concerns and a user-friendly workflow. The strongest documented design themes are guided input, controlled AI generation, editable output, multi-format export and deployment readiness. The supplied sample Employment Agreement demonstrates the intended output structure and the use of [TO BE COMPLETED] for missing facts.

For academic/project presentation

A concise presentation narrative is: **“LegalEase AI accepts structured legal inputs, validates them through FastAPI/Pydantic, generates a draft through Gemini or deterministic demo mode, sanitizes the result, lets the user edit and preview it, and exports the final draft as TXT, DOCX or PDF.”**